

Data Structures & Algorithms in C

CSA 0301

CO 1: Assessment 1

Name: Pranav A S

Register number: 192573001

Q1: Smart Healthcare System

Concepts: Data → Data Object → Data Structure → ADT → C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Patient {
    int id;
    char name[50];
    float temperature;
    struct Patient* next;
} Patient;

typedef struct {
    Patient* head;
    Patient* tail;
    int size;
} PatientQueueADT;

PatientQueueADT* createQueue() {
    PatientQueueADT* q = (PatientQueueADT*)malloc(sizeof(PatientQueueADT));
    q->head = q->tail = NULL;
    q->size = 0;
    return q;
}

void enqueuePatient(PatientQueueADT* q, int id, const char* name, float temp) {
    Patient* p = (Patient*)malloc(sizeof(Patient));
```

```

    p->id = id;
    strcpy(p->name, name);
    p->temperature = temp;
    p->next = NULL;

    if (q->tail == NULL) {
        q->head = q->tail = p;
    } else {
        q->tail->next = p;
        q->tail = p;
    }
    q->size++;
}

void processPatient(PatientQueueADT* q) {
    if (q->head == NULL) return;
    Patient* temp = q->head;
    printf("Processing Patient ID: %d | Name: %s | Temp: %.1f deg C\n", temp->id,
temp->name, temp->temperature);
    q->head = q->head->next;
    if (q->head == NULL) q->tail = NULL;
    free(temp);
    q->size--;
}

int main() {
    PatientQueueADT* ERQueue = createQueue();
    enqueuePatient(ERQueue, 101, "Alice", 38.5);
    enqueuePatient(ERQueue, 102, "Bob", 36.6);
    processPatient(ERQueue);
    processPatient(ERQueue);
    free(ERQueue);
    return 0;
}

```

Q2: Smart City Traffic Management

Concepts: Primitive → Non-Primitive → Linear (Queues) → Non-Linear (Graph)

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_INTERSECTIONS 4

typedef struct Graph {
    int adjMatrix[MAX_INTERSECTIONS][MAX_INTERSECTIONS];
} Graph;

typedef struct {
    int signalTimes[4];
    int front, rear;
} SignalQueue;

void initGraph(Graph* g) {
    for (int i = 0; i < MAX_INTERSECTIONS; i++)
        for (int j = 0; j < MAX_INTERSECTIONS; j++)
            g->adjMatrix[i][j] = 0;
}

void addRoad(Graph* g, int u, int v, int distance) {
    g->adjMatrix[u][v] = distance;
    g->adjMatrix[v][u] = distance;
}

int main() {
    Graph roadNetwork;
    initGraph(&roadNetwork);
    addRoad(&roadNetwork, 0, 1, 500);
    addRoad(&roadNetwork, 1, 2, 300);

    SignalQueue q = {{30, 45, 60, 20}, 0, 3};
```

```

    printf("Road distance between 0 and 1: %d meters\n",
roadNetwork.adjMatrix[0][1]);
    printf("Signal 1 Green Time: %d seconds\n", q.signalTimes[0]);
    return 0;
}

```

Q3: E-Commerce Platform

Concepts: Linear Sequential Limitations → Non-Linear Search (BST) & Recommendations

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct ProductNode {
    int productId;
    char name[50];
    struct ProductNode *left, *right;
} ProductNode;

ProductNode* createProduct(int id, const char* name) {
    ProductNode* p = (ProductNode*)malloc(sizeof(ProductNode));
    p->productId = id;
    strcpy(p->name, name);
    p->left = p->right = NULL;
    return p;
}

ProductNode* insertProduct(ProductNode* root, int id, const char* name) {
    if (root == NULL) return createProduct(id, name);
    if (id < root->productId) root->left = insertProduct(root->left, id, name);
    else if (id > root->productId) root->right = insertProduct(root->right, id, name);
    return root;
}

```

```

ProductNode* searchProduct(ProductNode* root, int id) {
    if (root == NULL || root->productId == id) return root;
    if (id < root->productId) return searchProduct(root->left, id);
    return searchProduct(root->right, id);
}

int main() {
    ProductNode* catalog = NULL;
    catalog = insertProduct(catalog, 105, "Laptop");
    catalog = insertProduct(catalog, 101, "Headphones");
    catalog = insertProduct(catalog, 110, "Smartphone");

    ProductNode* item = searchProduct(catalog, 101);
    if (item) printf("Product Found: %s\n", item->name);
    else printf("Product Not Found\n");

    return 0;
}

```

Q4: Ride-Sharing Driver Matching

Concepts: Algorithm → Input Size → Time Complexity → Performance

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

typedef struct {
    int driverId;
    int x, y;
} Driver;

int findNearestDriverLinear(Driver drivers[], int n, int userX, int userY) {
    int minIndex = -1;

```

```

double minDistance = 1e9;

for (int i = 0; i < n; i++) {
    double dist = sqrt(pow(drivers[i].x - userX, 2) + pow(drivers[i].y - userY, 2));
    if (dist < minDistance) {
        minDistance = dist;
        minIndex = i;
    }
}
return drivers[minIndex].driverId;
}

int main() {
    Driver drivers[3] = {{1, 10, 20}, {2, 3, 4}, {3, 50, 50}};
    int nearest = findNearestDriverLinear(drivers, 3, 0, 0);
    printf("Optimal Match Driver ID: %d\n", nearest);
    return 0;
}

```

Q5: Memory-Constrained Mobile Application

Concepts: Memory Usage → Auxiliary Space → Algorithm Efficiency

```

#include <stdio.h>

void processDataInPlace(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        arr[i] = arr[i] * 2;
    }
}

int main() {
    int sensorData[5] = {10, 20, 30, 40, 50};
    processDataInPlace(sensorData, 5);
    printf("Processed Value: %d\n", sensorData[0]);
}

```

```
    return 0;
}
```

Q6: Banking System ADT

Concepts: ADT → Operations → Interface → Implementation Decoupling

```
#include <stdio.h>
#include <stdlib.h>

typedef struct BankAccount BankAccount;

struct BankAccount {
    int accountNumber;
    double balance;
};

BankAccount* createAccount(int accNum, double initialBalance) {
    BankAccount* acc = (BankAccount*)malloc(sizeof(BankAccount));
    acc->accountNumber = accNum;
    acc->balance = initialBalance;
    return acc;
}

void deposit(BankAccount* acc, double amount) {
    if (amount > 0) acc->balance += amount;
}

int withdraw(BankAccount* acc, double amount) {
    if (acc->balance >= amount) {
        acc->balance -= amount;
        return 1;
    }
    return 0;
}
```

```

double getBalance(BankAccount* acc) {
    return acc->balance;
}

int main() {
    BankAccount* userAcc = createAccount(1001, 500.00);
    deposit(userAcc, 150.50);
    withdraw(userAcc, 200.00);
    printf("Current Balance: $%.2f\n", getBalance(userAcc));
    free(userAcc);
    return 0;
}

```

Q7: Hierarchical File System Traversal

Concepts: Recursion → Base Case → Recursive Case → Call Stack

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct FileNode {
    char name[50];
    int isFolder;
    struct FileNode* child;
    struct FileNode* sibling;
} FileNode;

FileNode* createNode(const char* name, int isFolder) {
    FileNode* node = (FileNode*)malloc(sizeof(FileNode));
    strcpy(node->name, name);
    node->isFolder = isFolder;
    node->child = node->sibling = NULL;
    return node;
}

```

```

}

void printFileSystem(FileNode* root, int depth) {
    if (root == NULL) return;

    for (int i = 0; i < depth; i++) printf(" ");
    printf("%s %s\n", root->isFolder ? "[DIR]" : "[FILE]", root->name);

    printFileSystem(root->child, depth + 1);
    printFileSystem(root->sibling, depth);
}

int main() {
    FileNode* root = createNode("C:", 1);
    root->child = createNode("Documents", 1);
    root->child->child = createNode("resume.pdf", 0);
    root->child->sibling = createNode("Pictures", 1);

    printFileSystem(root, 0);
    return 0;
}

```

Q8: Recursion vs Iteration Comparison

Concepts: Recursion (Call Stack $O(N)$) vs Iteration ($O(1)$ Space)

```

#include <stdio.h>

long long factorialRecursive(int n) {
    if (n <= 1) return 1;
    return n * factorialRecursive(n - 1);
}

long long factorialIterative(int n) {
    long long result = 1;

```

```

    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    int val = 10;
    printf("Recursive Result: %lld\n", factorialRecursive(val));
    printf("Iterative Result: %lld\n", factorialIterative(val));
    return 0;
}

```

Q9: Music Streaming Playlist

Concepts: Linked List for Dynamic Sequential Operations

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Song {
    char title[50];
    struct Song* next;
    struct Song* prev;
} Song;

typedef struct {
    Song* head;
    Song* tail;
    Song* current;
} Playlist;

Playlist* createPlaylist() {
    Playlist* pl = (Playlist*)malloc(sizeof(Playlist));

```

```

    pl->head = pl->tail = pl->current = NULL;
    return pl;
}

void addSong(Playlist* pl, const char* title) {
    Song* s = (Song*)malloc(sizeof(Song));
    strcpy(s->title, title);
    s->next = NULL;
    s->prev = pl->tail;

    if (pl->tail) pl->tail->next = s;
    else pl->head = s;

    pl->tail = s;
    if (!pl->current) pl->current = s;
}

void playNext(Playlist* pl) {
    if (pl->current && pl->current->next) {
        pl->current = pl->current->next;
        printf("Now Playing: %s\n", pl->current->title);
    }
}

int main() {
    Playlist* myPlaylist = createPlaylist();
    addSong(myPlaylist, "Song A");
    addSong(myPlaylist, "Song B");
    printf("Now Playing: %s\n", myPlaylist->current->title);
    playNext(myPlaylist);
    return 0;
}

```

Q10: Social Media Network Graph

Concepts: Graph Network → Adjacency List → Depth-First Search Traversal

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_USERS 10

typedef struct Node {
    int vertex;
    struct Node* next;
} Node;

typedef struct Graph {
    Node* adjLists[MAX_USERS];
    int visited[MAX_USERS];
} Graph;

Node* createNode(int v) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

void addFriendship(Graph* g, int src, int dest) {
    Node* newNode = createNode(dest);
    newNode->next = g->adjLists[src];
    g->adjLists[src] = newNode;

    newNode = createNode(src);
    newNode->next = g->adjLists[dest];
    g->adjLists[dest] = newNode;
}

void DFS(Graph* g, int vertex) {
```

```

g->visited[vertex] = 1;
printf("Visited User: %d\n", vertex);

Node* temp = g->adjLists[vertex];
while (temp) {
    int connectedVertex = temp->vertex;
    if (!g->visited[connectedVertex]) {
        DFS(g, connectedVertex);
    }
    temp = temp->next;
}
}

int main() {
    Graph* socialNetwork = (Graph*)malloc(sizeof(Graph));
    for (int i = 0; i < MAX_USERS; i++) {
        socialNetwork->adjLists[i] = NULL;
        socialNetwork->visited[i] = 0;
    }

    addFriendship(socialNetwork, 0, 1);
    addFriendship(socialNetwork, 0, 2);
    addFriendship(socialNetwork, 1, 3);

    DFS(socialNetwork, 0);
    return 0;
}

```

Q11: Searching Algorithms Comparison

Concepts: Linear Search $O(N)$ vs Binary Search $O(\log N)$

```
#include <stdio.h>

int linearSearch(int arr[], int n, int target, int* ops) {
    *ops = 0;
    for (int i = 0; i < n; i++) {
        (*ops)++;
        if (arr[i] == target) return i;
    }
    return -1;
}

int binarySearch(int arr[], int n, int target, int* ops) {
    int low = 0, high = n - 1;
    *ops = 0;
    while (low <= high) {
        (*ops)++;
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int dataset[8] = {10, 20, 30, 40, 50, 60, 70, 80};
    int opsLinear = 0, opsBinary = 0;

    linearSearch(dataset, 8, 70, &opsLinear);
```

```

binarySearch(dataset, 8, 70, &opsBinary);

printf("Linear Search Operations: %d\n", opsLinear);
printf("Binary Search Operations: %d\n", opsBinary);
return 0;
}

```

Q12: Ride-Sharing Integration Architecture

Concepts: ADTs → Linear Queue Request Handling

```

#include <stdio.h>
#include <stdlib.h>

typedef struct RequestNode {
    int userId;
    struct RequestNode* next;
} RequestNode;

typedef struct {
    RequestNode *front, *rear;
} RequestQueue;

void enqueueRequest(RequestQueue* q, int id) {
    RequestNode* temp = (RequestNode*)malloc(sizeof(RequestNode));
    temp->userId = id;
    temp->next = NULL;
    if (!q->rear) {
        q->front = q->rear = temp;
        return;
    }
    q->rear->next = temp;
    q->rear = temp;
}

```

```

int dequeueRequest(RequestQueue* q) {
    if (!q->front) return -1;
    RequestNode* temp = q->front;
    int id = temp->userId;
    q->front = q->front->next;
    if (!q->front) q->rear = NULL;
    free(temp);
    return id;
}

int main() {
    RequestQueue rideRequests = {NULL, NULL};
    enqueueRequest(&rideRequests, 501);
    enqueueRequest(&rideRequests, 502);

    printf("Allocated driver for User: %d\n", dequeueRequest(&rideRequests));
    return 0;
}

```

Q13: Library Management Hash Table

Concepts: Direct Access & Fast Retrieval via Hash Table

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define TABLE_SIZE 10

typedef struct Book {
    int bookId;
    char title[50];
    struct Book* next;
} Book;

```

```
Book* hashTable[TABLE_SIZE];

int hashFunction(int key) {
    return key % TABLE_SIZE;
}

void insertBook(int id, const char* title) {
    int index = hashFunction(id);
    Book* newBook = (Book*)malloc(sizeof(Book));
    newBook->bookId = id;
    strcpy(newBook->title, title);
    newBook->next = hashTable[index];
    hashTable[index] = newBook;
}

Book* searchBook(int id) {
    int index = hashFunction(id);
    Book* temp = hashTable[index];
    while (temp) {
        if (temp->bookId == id) return temp;
        temp = temp->next;
    }
    return NULL;
}

int main() {
    insertBook(1001, "Data Structures in C");
    insertBook(2001, "Operating Systems");

    Book* b = searchBook(1001);
    if (b) printf("Found Book: %s\n", b->title);
    return 0;
}
```

Q14: Real-Time Ticket Booking System

Concepts: Direct Access Array vs Linked List Allocation Efficiency

```
#include <stdio.h>

#define TOTAL_SEATS 5

typedef struct {
    int seatMap[TOTAL_SEATS];
} ArrayBookingSystem;

int bookSeat(ArrayBookingSystem* sys, int seatNumber) {
    if (seatNumber < 0 || seatNumber >= TOTAL_SEATS) return 0;
    if (sys->seatMap[seatNumber] == 0) {
        sys->seatMap[seatNumber] = 1;
        return 1;
    }
    return -1;
}

int main() {
    ArrayBookingSystem theater = {{0}};
    if (bookSeat(&theater, 2) == 1) {
        printf("Seat 2 successfully booked!\n");
    }
    return 0;
}
```

Q15: Continuous Weather Stream Monitoring

Concepts: Circular Buffer Stream Data Processing

```
#include <stdio.h>

#define BUFFER_SIZE 4
```

```

typedef struct {
    float temperatureBuffer[BUFFER_SIZE];
    int head;
    int count;
} WeatherTracker;

void addReading(WeatherTracker* tracker, float temp) {
    tracker->temperatureBuffer[tracker->head] = temp;
    tracker->head = (tracker->head + 1) % BUFFER_SIZE;
    if (tracker->count < BUFFER_SIZE) tracker->count++;
}

void printReadings(WeatherTracker* tracker) {
    printf("Recent Readings: ");
    for (int i = 0; i < tracker->count; i++) {
        printf("%.1f ", tracker->temperatureBuffer[i]);
    }
    printf("\n");
}

int main() {
    WeatherTracker tracker = {{0.0f}, 0, 0};
    addReading(&tracker, 28.5);
    addReading(&tracker, 29.1);
    addReading(&tracker, 30.0);
    addReading(&tracker, 31.2);
    addReading(&tracker, 27.8);

    printReadings(&tracker);
    return 0;
}

```